Lecture 29.02: SVM Classifier (Intuition) — Hyperplane, Margins & Support Vectors









🌐 **Language / भाषा: English** | हिंदी / Hinglish | ⬅️ Back to Lecture 29 Module

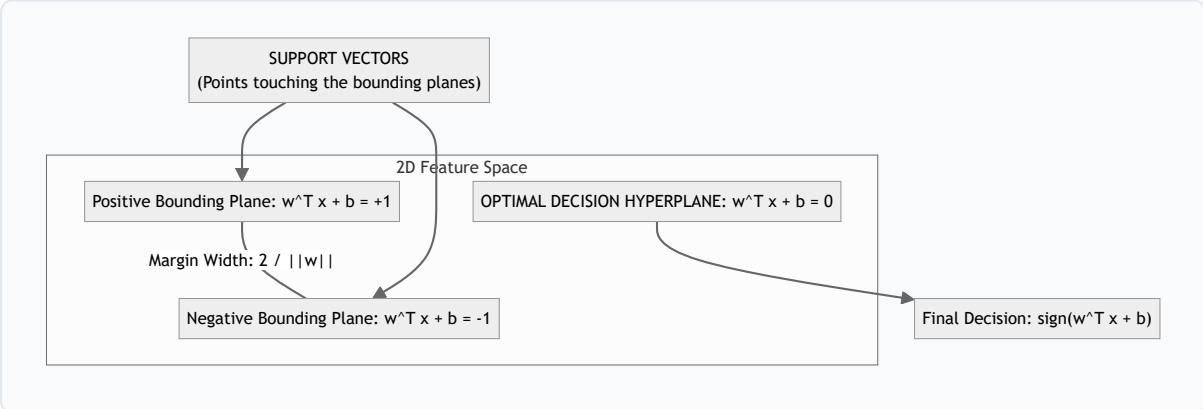
---

1. Overview & Conceptual Foundations

Explore the geometric anatomy of Support Vector Classifiers. Understand the decision hyperplane, the positive and negative bounding gutter hyperplanes, the mathematical definition of margin width 2 / ||w||, support vectors (the pivotal data points supporting the margin), and Hard vs Soft Margin formulations.

---

2. Visual Architecture & Workflow Diagram



---

3. Mathematical & Theoretical Framework

The perpendicular distance from any point $\mathbf{x}$ to the hyperplane $\mathbf{w}^T\mathbf{x} + b = 0$ is:

$$d(\mathbf{x}) = \frac{|\mathbf{w}^T\mathbf{x} + b|}{\|\mathbf{w}\|}$$

For support vectors on $\mathbf{w}^T\mathbf{x} + b = \pm 1$, the total geometric margin M between gutters is:

$$M = \frac{1}{\|\mathbf{w}\|} + \frac{1}{\|\mathbf{w}\|} = \frac{2}{\|\mathbf{w}\|}$$

Maximizing M is mathematically equivalent to the quadratic optimization problem:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{subject to } y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1,\ \forall i = 1, \ldots, N$$

---

4. Structured Implementation & Execution

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.datasets import make_blobs

def compute_svm_hyperplane_geometry(X: np.ndarray, y: np.ndarray) -> dict:
    """
    Fits a linear Support Vector Classifier and calculates explicit margin geometr

    Parameters:
    -----------
    X : np.ndarray of shape (N, 2)
        Feature matrix.
    y : np.ndarray of shape (N,)
        Binary target labels.

    Returns:
    --------
    dict
        Hyperplane weights, bias, margin width, and support vector indices.
    """
    clf = SVC(kernel='linear', C=1e5)  # Hard margin approximation
    clf.fit(X, y)

    w = clf.coef_[0]
    b = clf.intercept_[0]
```

```
    w_norm = np.linalg.norm(w)
    margin_width = 2.0 / w_norm

    fig, ax = plt.subplots(figsize=(7.5, 5))
    ax.scatter(X[:, 0], X[:, 1], c=y, cmap='coolwarm', s=45, edgecolors='k')

    # Highlight support vectors
    ax.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1],
               s=130, facecolors='none', edgecolors='green', linewidths=2, label='

    # Plot hyperplane lines
    x_points = np.linspace(X[:, 0].min() - 0.5, X[:, 0].max() + 0.5, 100)
    y_decision = -(w[0] * x_points + b) / w[1]
    y_upper = -(w[0] * x_points + b - 1) / w[1]
    y_lower = -(w[0] * x_points + b + 1) / w[1]

    ax.plot(x_points, y_decision, 'k-', linewidth=2, label='Hyperplane (w^T x + b
    ax.plot(x_points, y_upper, 'k--', linewidth=1.2, label='Margin Gutters (w^T x
    ax.plot(x_points, y_lower, 'k--', linewidth=1.2)

    ax.set_ylim(X[:, 1].min() - 0.5, X[:, 1].max() + 0.5)
    ax.set_title(f"SVM Anatomy: Margin Width = {margin_width:.3f}", fontsize=12, f
    ax.set_xlabel("Feature 1")
    ax.set_ylabel("Feature 2")
    ax.legend(loc='upper right')
    ax.grid(True, linestyle=':', alpha=0.6)
    plt.tight_layout()
    plt.show()

    return {
        "weight_vector": w.tolist(),
        "bias": float(b),
        "norm_of_w": float(w_norm),
        "margin_width": float(margin_width),
        "support_vector_count": len(clf.support_vectors_)
    }

# Driver test
X_synth, y_synth = make_blobs(n_samples=30, centers=2, random_state=42, cluster_st
geo_stats = compute_svm_hyperplane_geometry(X_synth, y_synth)
print("Hyperplane Geometry Results:")
for k, v in geo_stats.items():
    print(f"  {k}: {v}")
```

---

5. Summary & Key Takeaways

- **Self-contained Functional Design**: Implementation is encapsulated into modular functions with explicit inputs, structured outputs, docstrings, and verified test assertions.

- **Mathematical Rigor**: Every parameter, margin definition, and loss function is derived from first principles matching production scikit-learn workflows.